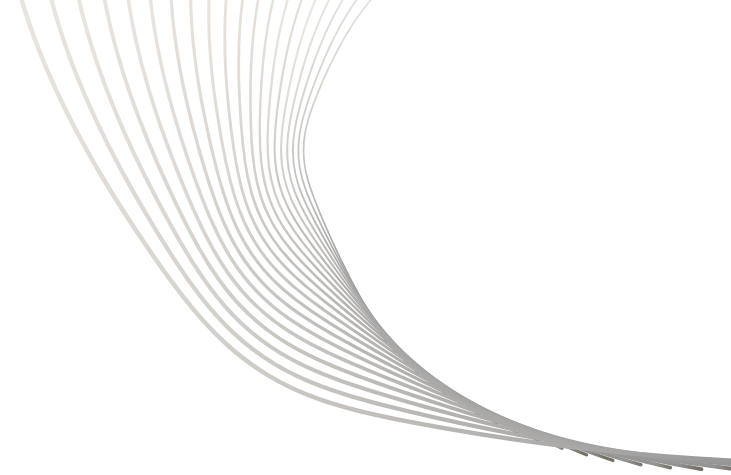
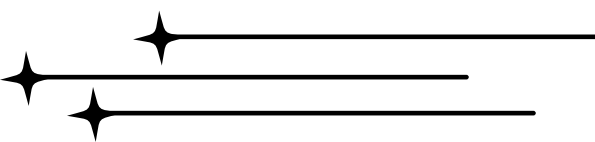




大綱

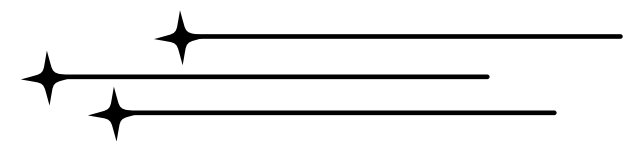
一、專題摘要

四、系統設計

二、研究動機

五、實機demo

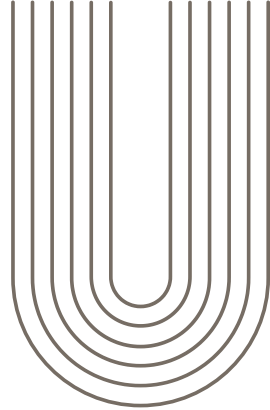
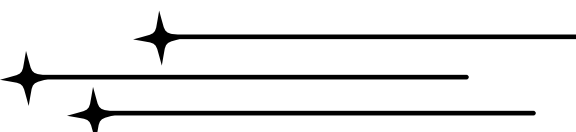
三、系統架構



專題摘要

結合大型語言模型（LLM）與檢索增強生成技術（RAG），系統前端採用 HTML/CSS/Javascript 與 socket 技術構建即時聊天室與行事曆介面，後端則以 Node.js/MongoDB 以及 Python 串接 GPT API（gpt-4o-mini），負責資料儲存、格式轉換、排程語句判斷以及核心排程運算。



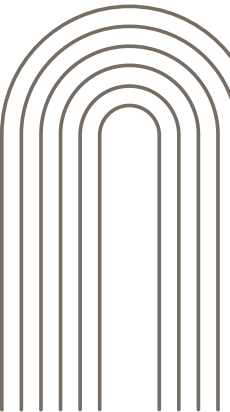


研究動機



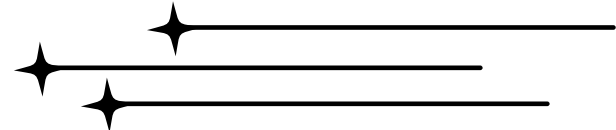
靈感來源

現實生活中，若人們不瞭解自身作業習慣，盲目排程恐導致達成率較低，並引發後續行程的連鎖延遲。因此本系統透過 **AI** 智慧偵測與排程詢問，提供個人化時間建議，協助使用者建立精準節奏，提升工作效率。

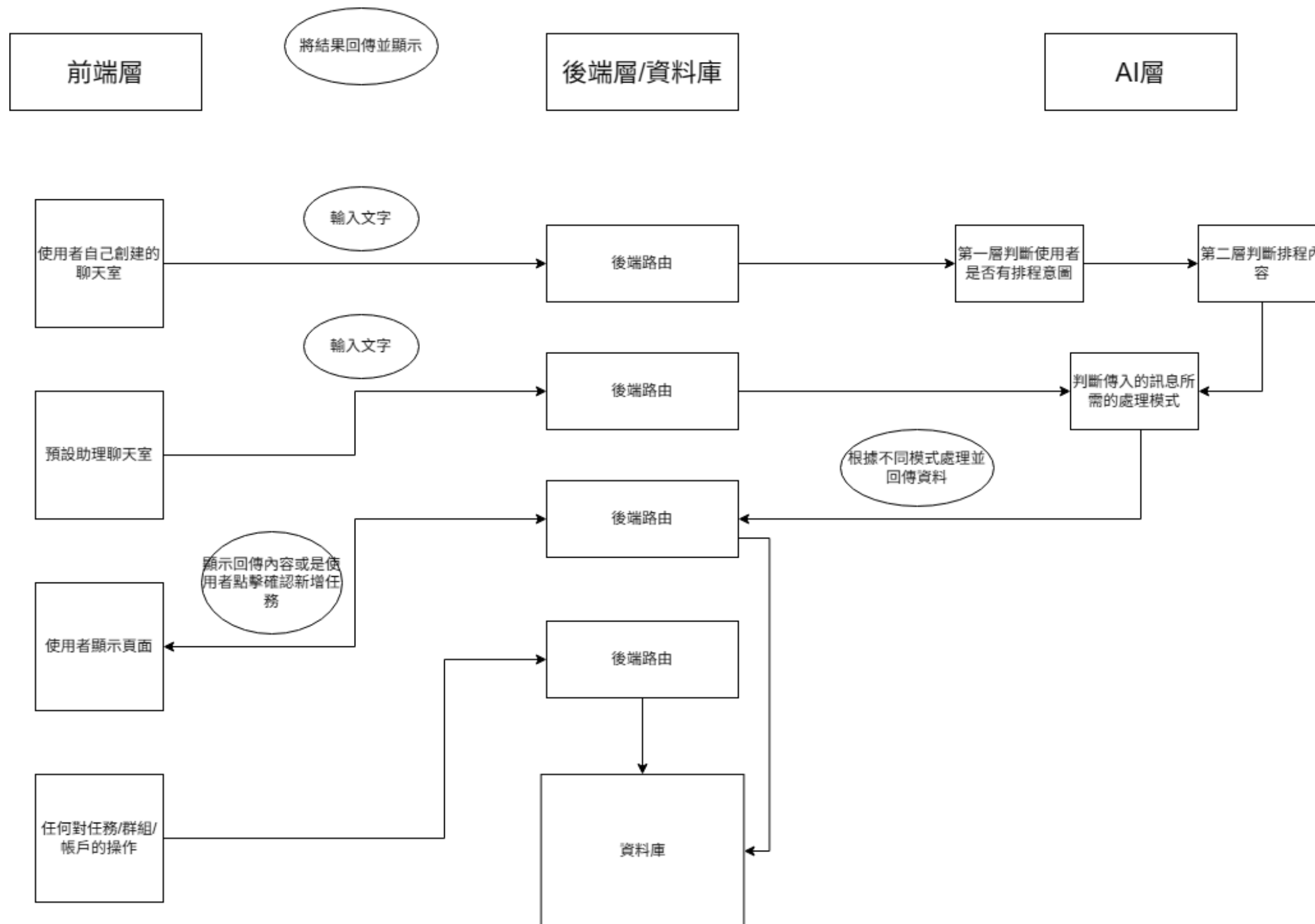


實務應用

在快節奏的生活中，傳統排程系統過度依賴手動規劃。但如果有一套 **AI** 排程系統分析輔助使用者的需求，並根據歷史紀錄推薦適合的時段，那這將有助於使用者提升時間的管理效率。



系統架構



前端層

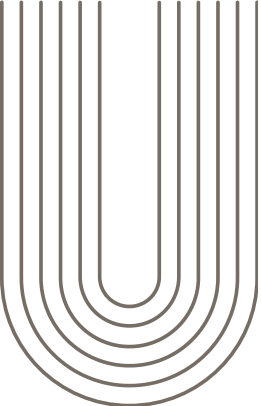
AI 助理聊天室 + 群組聊天室、
顯示排程建議、使用者偏好表單

後端層

Node.js 路由、MongoDB 資料存取、Python AI 模組呼叫

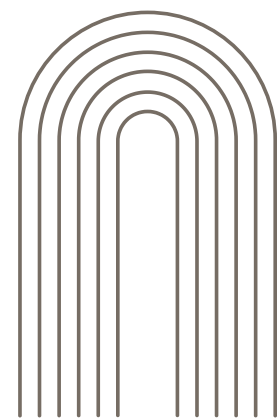
AI 決策層

兩階段篩選 + 多模式分類 +
RAG 向量檢索

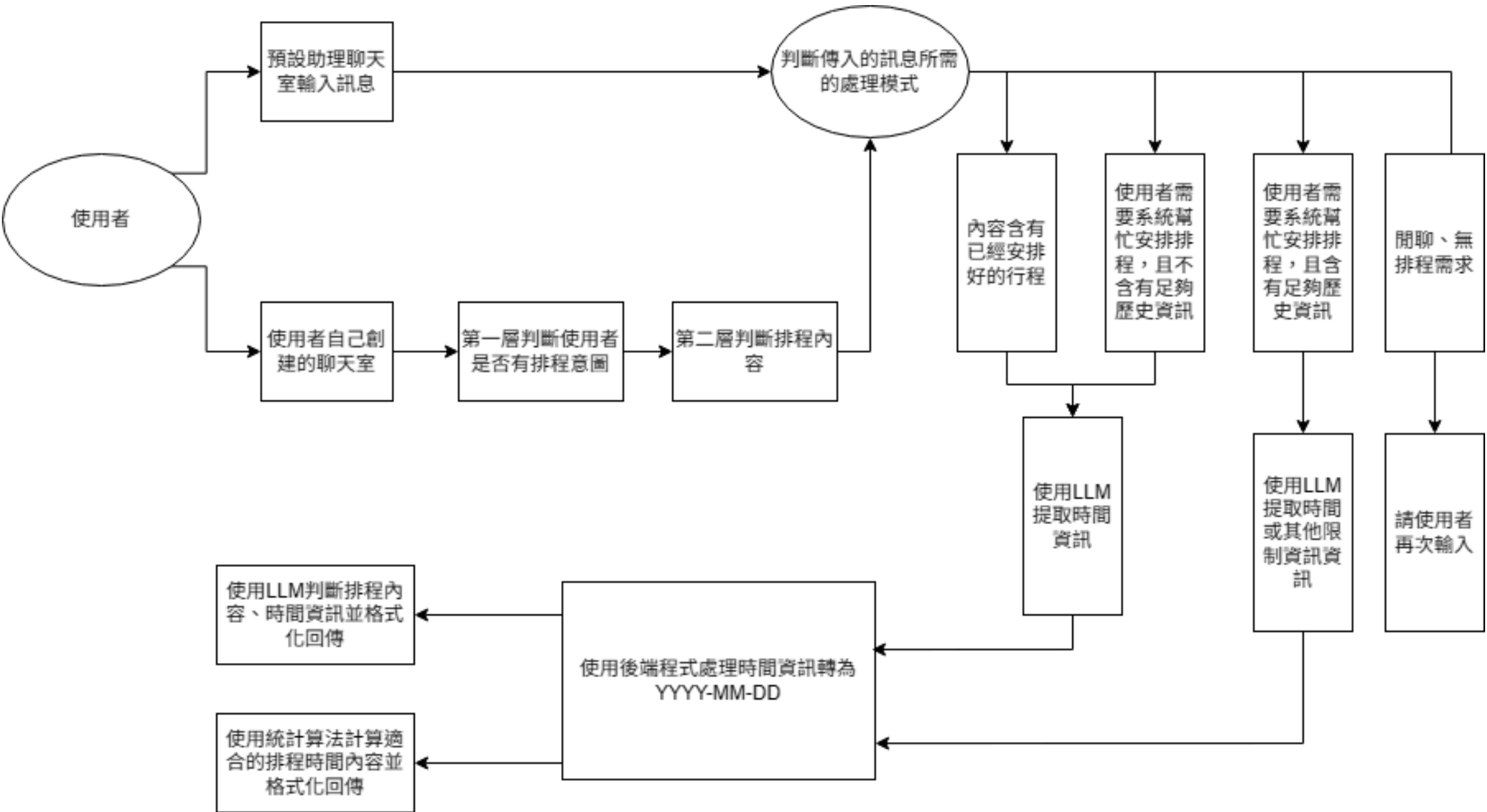


系統架構 - 資料庫

1. **Account (帳戶設定)**：儲存使用者基本資訊、關聯群組、擁有的任務 ID，以及透過表單蒐集而來的「個人時間偏好設定」。
2. **Group (群組紀錄)**：儲存該群組的基本資訊、成員名單，以及群組內的對話歷史紀錄，作為推論上下文之用。
3. **Task (任務內容)**：儲存任務的基本資訊，紀錄任務所有者、日期、標題、內容、耗時，以及「任務執行成效」（如是否完成、完成時的疲勞度、實際執行時間等數據），這些任務資訊將作為演算法的參考與統計資料。



系統設計流程



兩大輸入管道

預設助理聊天室：

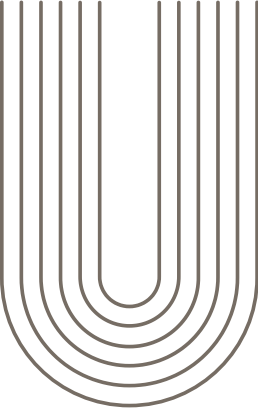
直接進入 **AI** 決策層進行意圖判斷

使用者自建聊天室：

經兩階段意圖判斷後進入決策節點

四種處理模式

依排程明確度與歷史資料量做處理



系統設計 - Mode 1

情境

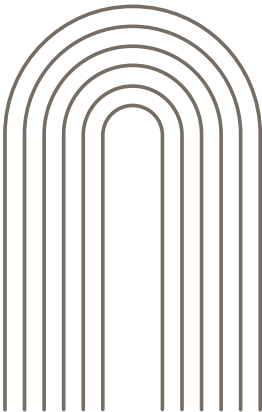
使用者提供清楚且具體的時間與任務目的，像是(明天)(下午一點)(要開會)、(下禮拜一)(晚上七點)(要健身)。

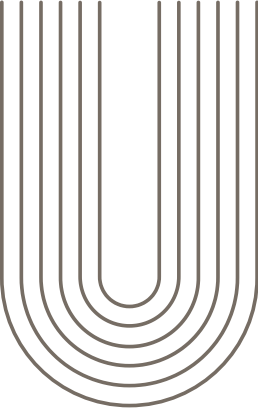
處理方式

第一層LLM：提取明天、下禮拜等等要處理的日期文字

程式邏輯：計算明天、下禮拜轉為實際日期區間(YYYY-MM-DD,YYYY-MM-DD)

第二層LLM：提取最終(日期)(時間)(目的)





系統設計 - Mode 2

情境

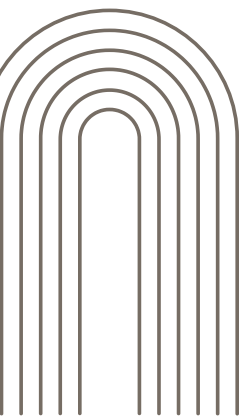
未指定明確時間像是明天下午要開會、我要讀書，且使用者歷史紀錄已經超過 10 筆使用的統計算法。避免在資料過少時開始使用統計的方式，進而連帶影響後面排程資料量繼續偏頗。

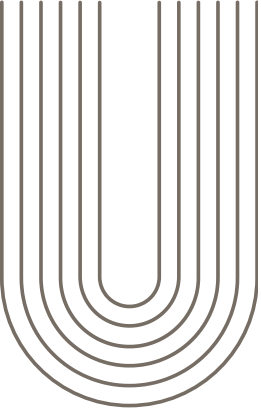
處理方式

提取任務類別：像是工作、學習等等

統計算法：綜合過往習慣、向量檢索匹配來調整權重，計算每個時間段的得分

LLM：與使用Mode1的方法抽取日期、時間、目的，並轉為符合的格式





系統設計 - Mode 3

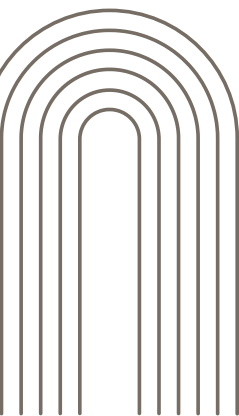
情境

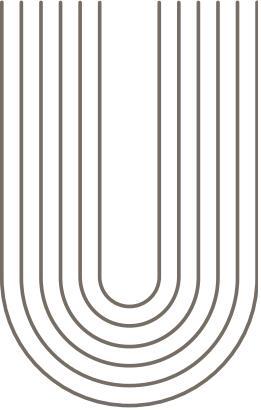
與Mode 2邏輯相似，但當使用者歷史資料不足時，系統無法單純依賴統計推論。此時會結合使用者事先填寫的偏好表單，以及當前對話內容，交由 LLM 進行綜合推論，以避免我們透過算法進行過度推論。

處理方式

第一層LLM：提供使用者的習慣表單、文字、歷史紀錄，判斷產生排程建議

第二層LLM：與使用Mode1的方法抽取日期、時間、目的，並轉為符合的格式





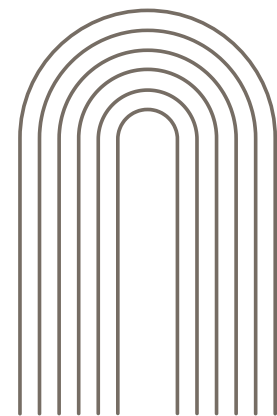
系統設計 - Mode 4

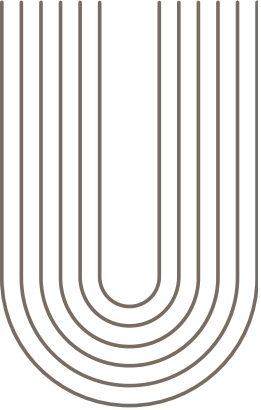
情境

若系統判定使用者輸入內容與排程無直接關聯，則不進行任務建立，並引導使用者重新輸入具體需求，以避免系統行程資料遭到無意義資訊干擾。

處理方式

系統判斷為無關排程的一般閒聊，不產出任何任務，請使用者重新輸入需求。





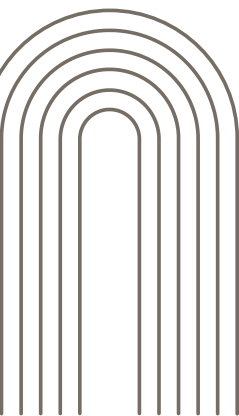
系統設計 - 普通聊天室兩層

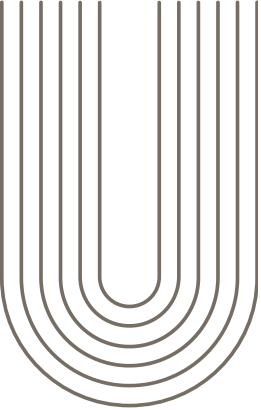
問題

單用一層 LLM 判斷意圖以及排程任務細節資訊，LLM 容易當作每次的輸入都是有排程意圖，硬去提取出可能符合需求的内容，沒辦法有效判斷當前語句的意圖，且當排程資訊密集時，輸出效果會非常差。

處理方式

使用兩層 LLM 依序判斷使用者輸入，第一層僅用於判斷輸入是否有排程意圖，而第二層則專注於提取排程任務細節資訊。





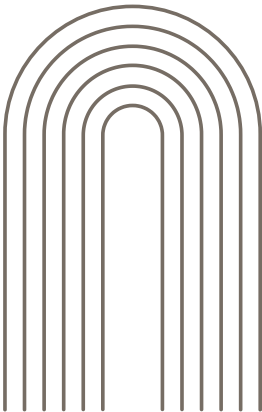
系統設計 - 普通聊天室兩層

第一階段

排程意圖偵測系統提供少量三句的聊天室歷史紀錄加上使用者發送的訊息，以及 **RAG** 的語句判斷情境，來判定該語句是否具有潛在排程意圖，並僅回傳 **YES** 或 **NO** 作為篩選結果。

第二階段

排程內容萃取當第一階段判定為 **YES** 時，結合聊天室十句歷史對話，以及對應的使用者**ID**、當前的目標語句，以提取與語句相關的排程訊息。



系統設計 - 排程演算法

計算基礎分數

依歷史活動紀錄與任務類型分配得分

調整舒適與疲勞

套用專注期與疲勞懲罰

最終分數公式

最終分數 = 基礎分數 + 舒適度加分 - 疲勞懲罰 + RAG調整 - 日期使用懲罰

產生候選時段

每日48個時間區塊生成候選

RAG 記憶加權

根據歷史經驗動態調整權重

系統設計 - 演算法細項

基礎分數計算機制

根據使用者的歷史行為資料，建立兩類時間分布，一為整體活動時間分布，反映使用者平常在哪些時段較常安排任務，二為任務類型時間分布，反映特定類型任務（像是工作、學習、運動）過去較常出現的時間位置，演算法將兩者整合為基礎分數，使系統不只考慮「常用時段」，也重視「適合該任務類型的時段」。

候選時段選擇與衝突避免

為避免任務過度集中於同一天，設計了日期使用懲罰，若某一日期已排入較多任務，則會降低其最終分數，另一方面，系統也會持續更新已使用的時間區塊，使後續任務不會與既有安排重疊，確保整體排程的可執行性與合理性。

整體系統架構介紹 - 演算法細項

舒適度與疲勞成本參數

LLM 會根據使用者的對話判斷使用者排程時的狀態並設定參數，分別為：

輕鬆 (relaxed)：提升舒適度並提高疲勞成本

疲累 (push)：降低疲勞懲罰並降低舒適度

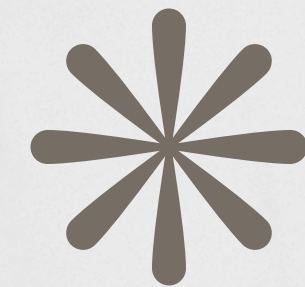
普通 (normal)：不調整任何參數，維持預設

RAG 記憶加權機制

從使用者過去相似情境的成功經驗中，檢索可用的調整資訊：

一、動態修正參數權重，包含提高舒適度權重、調整疲勞倍率

二、若過往成功紀錄集中於特定時間區段，則可直接對對應 **block** 進行加分或扣分，而檢索到的記錄日期距離現在越遠則加分以及扣分會遞減



實機DEMO



感謝聆聽

